Before we look at high-level languages, we make a detour to look at object-oriented design. Object-oriented design is another way of looking at the design process, which views a program from the standpoint of data rather than tasks. Because the functionality associated with this design process is often incorporated into high-level programming languages, we need to understand this design process before looking at specific high-level languages.

? **It's in! No, it's out!**

Have you ever watched a tennis match on television in which a player asks for confirmation of a line call? On a big screen, you can see the path of the ball and the point of impact that shows whether the ball was in or out. How do they do it? By computer, of course. One system uses four high-speed digital cameras with computer software that can track a ball, determine its trajectory, and map its impact point. These cameras are connected to one another and to a main computer using wireless technology.

9.1 Object-Oriented Methodology

We cover top-down design first because it more closely mirrors the way humans solve problems. As you saw earlier in this book, a top-down solution produces a hierarchy of tasks. Each task or named action operates on the data passed to it through its parameter list to produce the desired output. The tasks are the focus of a top-down design. Object-oriented design, by contrast, is a problem-solving methodology that produces a solution to a problem in terms of self-contained entities called *objects*, which are composed of both data and operations that manipulate the data. Object-oriented design focuses on the objects and their interactions within a problem. Once all of the objects within the problem are collected together, they constitute the solution to the problem.

In this process, Polya's principles of problem solving are applied to the data rather than to the tasks.

Object Orientation

Data and the algorithms that manipulate the data are bundled together in the object-oriented view, thus making each object responsible for its own manipulation (behavior). Underlying object-oriented design (OOD) are the concepts of *classes* and *objects*.

Object An entity or thing that is relevant in the context of a problem

Object class (class) A description of a group of objects with similar properties and behaviors

An **object** is an entity that makes sense within the context of the problem. For example, if the problem relates to information about students, a student would be a reasonable object in the solution. A group of similar objects is described by an **object class** (or **class** for short). Although no two students are identical, students do have properties (data) and behaviors (actions) in common. Students are humans who attend courses at a school (at least most of the time). Therefore, students would be a class. The word *class* refers to the idea of classifying objects into related groups and describing their common characteristics. That is, a class describes the properties and behaviors that objects of the class exhibit. Any particular object is an *instance* (concrete example) of the class.

Object-oriented problem solving involves isolating the classes within the problem. Objects communicate with one another by sending messages (invoking one another's subprograms). A class contains **fields** that represent the properties of the class. For example, a field of the class representing a student might be the student's name or ID number. A **method** is a named algorithm that manipulates the data values in the object. A class is a representation for what an object is (fields) and how it behaves (methods).

Fields Named properties of a class

Method A named algorithm that defines one aspect of the behavior of a class

Design Methodology

The decomposition process that we present involves four stages. *Brainstorming* is the stage in which we make a first pass at determining the classes in the problem. *Filtering* is the stage in which we go back over the proposed classes determined in the brainstorming stage to see if any can be combined or if any are missing. Each class that survives the filtering stage is examined more carefully in the next stage.

Scenarios is the stage in which the behavior of each class is determined. Because each class is responsible for its own behavior, we call these behaviors *responsibilities*. In this stage, "what if" questions are explored to be sure that all situations are examined. When all of the responsibilities of each class have been determined, they are recorded, along with the names of any other classes with which the class must collaborate (interact) to complete its responsibility.

Responsibility algorithms is the last stage, in which the algorithms are written for the responsibilities for each of the classes. A notation device called a *CRC card* is a handy way to record the information about each class at this stage.

Let's look at each of these stages in a little more detail.

Brainstorming

What is brainstorming? The dictionary defines it as a group problem-solving technique that involves the spontaneous contribution of ideas from all members of the group.[1] Brainstorming brings to mind a movie or TV show where a group of bright young people tosses around ideas about an advertising slogan for the latest revolutionary product. This picture seems at odds with the traditional picture of a computer analyst working alone in a closed, windowless office for days who finally jumps up shouting, "Ah ha!" As computers have gotten more powerful, the problems that can be solved have become more complex, and the picture of the genius locked in a windowless room has become obsolete. Solutions to

complex problems need new and innovative solutions based on collective "Ah ha!"s—not the work of a single person.

In the context of object-oriented problem solving, brainstorming is a group activity designed to produce a list of possible classes to be used to solve a particular problem. Just as the people brainstorming to create an advertising slogan need to know something about the product before the session begins, brainstorming for classes requires that the participants know something about the problem. Each team member should enter the brainstorming session with a clear understanding of the problem to be solved. No doubt during the preparation, each team member will have generated his or her own preliminary list of classes.

Although brainstorming is usually a group activity, you can practice it by yourself on smaller problems.

Filtering

Brainstorming produces a tentative list of classes. The next phase is to take this list and determine which are the core classes in the problem solution. Perhaps two classes on the list are actually the same thing. These duplicate classes usually arise because people within different parts of an organization use different names for the same concept or entity. Also, two classes in the list may have many common attributes and behaviors that can be combined.

Some classes might not actually belong in the problem solution. For example, if we are simulating a calculator, we might list the user as a possible class. In reality, the user is not part of the internal workings of the simulation as a class; the user is an entity outside the problem who provides input to the simulation. Another possible class might be the *on* button. A little thought, however, shows that the *on* button is not part of the simulation; rather, it is what starts the simulation program running.

As the filtering is completed, the surviving list of classes is passed onto the next stage.

Scenarios

The goal of the scenarios phase is to assign responsibilities to each class. Responsibilities are eventually implemented as subprograms. At this stage we are interested only in *what* the tasks are, not in how they might be carried out.

Two types of responsibilities exist: what a class must know about itself (knowledge) and what a class must be able to do (behavior). A class *encapsulates* its data (knowledge) such that objects in one class cannot directly

access data in another class. **Encapsulation** is the bundling of data and actions so that the logical properties of the data and actions are separated from the implementation details. Encapsulation is a key to abstraction. At the same time, each class has the responsibility of making data (knowledge) available to other classes that need it. Therefore, each class has a responsibility to know the things about itself that others need to be able to get. For example, a student class should "know" its name and address, and a class that uses the student class should be able to "get" this information. These responsibilities are usually named with "Get" preceding the name of the data—for example, *GetName* or *GetEmailAddress*. Whether the email address is kept in the student class or whether the student class must ask some other class to access the address is irrelevant at this stage: The important fact is that the student class knows its own email address and can return it to a class that needs it.

Encapsulation
Bundling data and actions so that the logical properties of data and actions are separated from the implementation details

The responsibilities for behavior look more like the tasks we described in top-down design. For example, a responsibility might be for the student class to calculate its grade point average (GPA). In top-down design, we would say that a task is to calculate the GPA given the data. In object-oriented design, we say that the student class is responsible for calculating its own GPA. The distinction is both subtle and profound. The final code for the calculation may look the same, but it is executed in different ways. In a program based on a top-down design, the program calls the subprogram that calculates the GPA, passing the student object as a parameter. In an object-oriented program, a message is sent to the object of the class to calculate its GPA. There are no parameters because the object to which the message is sent knows its own data.

The name for this phase gives a clue about how you go about assigning responsibilities to classes. The team (or an individual) describes different processing scenarios involving the classes. Scenarios are "what if" scripts that allow participants to act out different situations or an individual to think through them.

The output from this phase is a set of classes with each class's responsibilities assigned, perhaps written on a CRC card. The responsibilities for each class are listed on the card, along with the classes with which a responsibility must collaborate.

Responsibility Algorithms

Eventually, algorithms must be written for the responsibilities. Because the problem-solving process focuses on data rather than actions in the object-oriented view of design, the algorithms for carrying out responsibilities

tend to be fairly short. For example, the knowledge responsibilities usually just return the contents of one of an object's variables or send a message to another object to retrieve it. Action responsibilities are a little more complicated, often involving calculations. Thus the top-down method of designing an algorithm is usually appropriate for designing action-responsibility algorithms.

Final Word

To summarize, top-down design methods focus on the *process* of transforming the input into the output, resulting in a hierarchy of tasks. Object-oriented design focuses on the *data objects* that are to be transformed, resulting in a hierarchy of objects. Grady Booch puts it this way: "Read the specification of the software you want to build. Underline the verbs if you are after procedural code, the nouns if you aim for an object-oriented program."[2]

We propose that you circle the nouns and underline the verbs as a way to begin. The nouns become objects; the verbs become operations. In a top-down design, the verbs are the primary focus; in an object-oriented design, the nouns are the primary focus.

Now, let's work through an example.

Example

Problem

Create a list that includes each person's name, telephone number, and email address. This list should then be printed in alphabetical order. The names to be included in the list are on scraps of paper and business cards.

Brainstorming and Filtering

Let's try circling the nouns and underlining the verbs.

Create a list that includes each person name, telephone number, and email address. This list should then be printed in alphabetical order. The names to be included in the list are on scraps of paper and business cards.

The first pass at a list of classes would include the following:

list
name
telephone number
email address
list
order
names
list
scraps
paper
cards

Three of these classes are the same: The three references to *list* all refer to the container being created. *Order* is a noun, but what is an *order class*? It actually describes how the list class should print its items. Therefore, we discard it as a class. *Name* and *names* should be combined into one class. *Scraps*, *paper*, and *cards* describe objects that contain the data in the real world. They have no counterpart within the design. Our filtered list is shown below:

list
name
telephone number
email address

The verbs in the problem statement give us a headstart on the responsibilities: *create*, *print*, and *include*. Like *scraps*, *paper*, and *cards*, *include* is an instruction to someone preparing the data and has no counterpart within the design. However, it does indicate that we must have an object that inputs the data to be put on the list. Exactly what is this data? It is the name, telephone number, and email address of each person on the list. But this train of thought leads to the discovery that we have missed a major clue in the problem statement. A possessive noun, *person's*, actually names a major class; name, telephone number, and email address are classes that help define (are contained within) a person class.

Now we have a design choice. Should the *person* class have a responsibility to input its own data to initialize itself, or should we create another class that does the input and sends the data to initialize the *person* class? Let's have the *person* class be responsible for initializing itself. The *person* class should also be responsible for printing itself.

Does the *person* class collaborate with any other class? The answer to this question depends on how we decide to represent the data in the *person* class. Do we represent *name*, *telephone number*, and *email address* as simple data items within the *person* class, or do we represent each as its own class? Let's represent *name* as a class with two data items, *firstName* and *lastName*, and have the others be string variables in the class *person*. Both classes *person* and *name* must have knowledge responsibilities for their data values. Here are the CRC cards for these classes.

Class Name: Person	Superclass:	Subclasses:
Responsibilities	**Collaborations**	
Initialize itself (name, telephone, email)	Name, String	
Print	Name, String	
GetEmail	String	
GetName	Name, String	
GetTelephone	String	

Class Name: Name	Superclass:	Subclasses:
Responsibilities	**Collaborations**	
Initialize itself (firstName, lastName)	String	
Print itself	String	
GetFirstName	String	
GetLastName	String	

What about the list object? Should the list keep the items in alphabetical order, or should it sort the items before printing them? Each language in which we might implement this design has a library of container classes available for use. Let's use one of these classes, which keeps the list in alphabetical order. This library class should also print the list. We

?

Beware of pirated software

A 2013 study analyzed 270 websites and peer-to-peer networks, 108 software downloads, 155 CDs or DVDs, 2077 consumer interviews, and 258 interviews of IT managers around the world. The results showed that of the counterfeit software that does not ship with the computer, 45% is downloaded from the Internet. Of software downloaded from websites or peer-to-peer networks, 78% include some type of spyware and 36% contain Trojans and adware.[3]

can create a CRC card for this class, but mark that it most likely will be implemented using a library class.

Class Name: SortedList (from library)	Superclass:	Subclasses:
Responsibilities	Collaborations	
Insert (person)	Person	
Print itself	Person	

By convention, when a class reaches the CRC stage, we begin its identifier with an uppercase letter.

Responsibility Algorithms

Person Class There are two responsibilities to be decomposed: initialize and print. Because Name is a class, we can just let it initialize and print itself. We apply a subprogram (method) to an object by placing the object name before the method name with a period in between.

Initialize

```
name.initialize()
Write "Enter phone number; press return."
Get telephone number
Write "Enter email address; press return."
Get email address
```


Print

```
name.print()
Write "Telephone number: ", telephoneNumber
Write "Email address: ", emailAddress
```

Name Class This class has the same two responsibilities: initialize and print. However, the algorithms are different. For the initialize responsibility, the user must be prompted to enter the name and the algorithm must read the name. For the print responsibility, the first and last names must be output with appropriate labels.

Initialize

"Enter the first name; press return."
Read firstName
"Enter the last name; press return."
Read lastName

Print

Print "First name: ", firstName
Print "Last name: ", lastName

We stop the design at this point. Reread the beginning of Chapter 7, where we discuss problem solving and the top-down design process. A top-down design produces a hierarchical tree with tasks in the nodes of the tree. The object-oriented design produces a set of classes, each of which has responsibilities for its own behavior. Is one better than the other? Well, the object-oriented design creates classes that might be useful in other contexts. Reusability is one of the great advantages of an object-oriented design. Classes designed for one problem can be used in another problem, because each class is self-contained; that is, each class is responsible for its own behavior.

You can think of the object-oriented problem-solving phase as mapping the objects in the real world into classes, which are descriptions of the categories of objects. The implementation phase takes the descriptions of the categories (classes) and creates instances of the classes that simulate

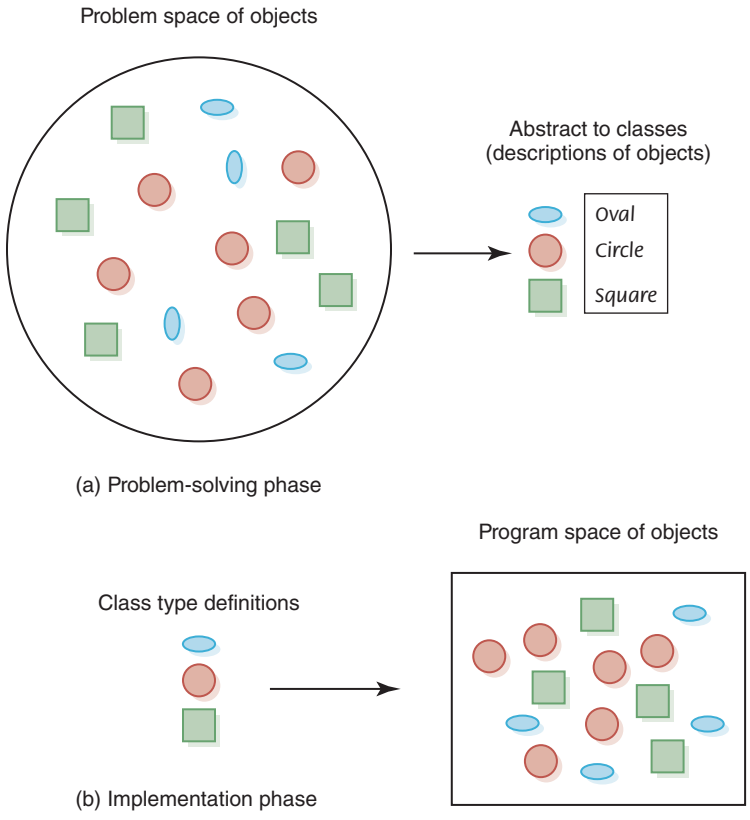

FIGURE 9.1 Mapping of a problem into a solution

the objects in the problem. The interactions of the objects in the program simulate the interaction of the objects in the real world of the problem. FIGURE 9.1 summarizes this process.

9.2 Translation Process

Recall from Chapter 6 that a program written in assembly language is input to the *assembler*, which translates the assembly-language instructions into machine code. The machine code, which is the output from the assembler, is then executed. With high-level languages, we employ other software tools to help with the translation process. Let's look at the basic function of these tools before examining high-level languages.